Project Report: Smart Hospital & Resource Allocation System

Course: CSC 1012 Introduction to Computer Programming

1. Introduction

The Smart Hospital & Resource Allocation System is a menu-driven C programming application developed to simulate basic hospital patient management and resource allocation.
The system manages patient registration, emergency priority levels, medical specialties, ward admissions, bed allocation, waiting-time estimation, billing calculations, patient prioritization, and hospital reports.
The project was developed using fundamental C programming concepts such as arrays, two-dimensional arrays, functions, conditional statements, loops, sorting algorithms, mathematical calculations, strings, and file handling.

2. Project Objective

The main objective of this project is to design and implement a Smart Hospital system that can manage patient information and hospital resources efficiently.
The system provides the following main functions:

- Patient registration
- Patient information display
- Hospital bed status management
- Automatic bed allocation
- Waiting-time calculation
- Emergency surcharge calculation
- Ward stay cost calculation
- Patient bill generation
- Age-based subsidy calculation
- Priority patient sorting
- Reports and analytics
- File-based record storage

3. System Architecture

The system follows a modular, menu-driven architecture.
The main program displays a menu and allows the user to select different hospital operations.

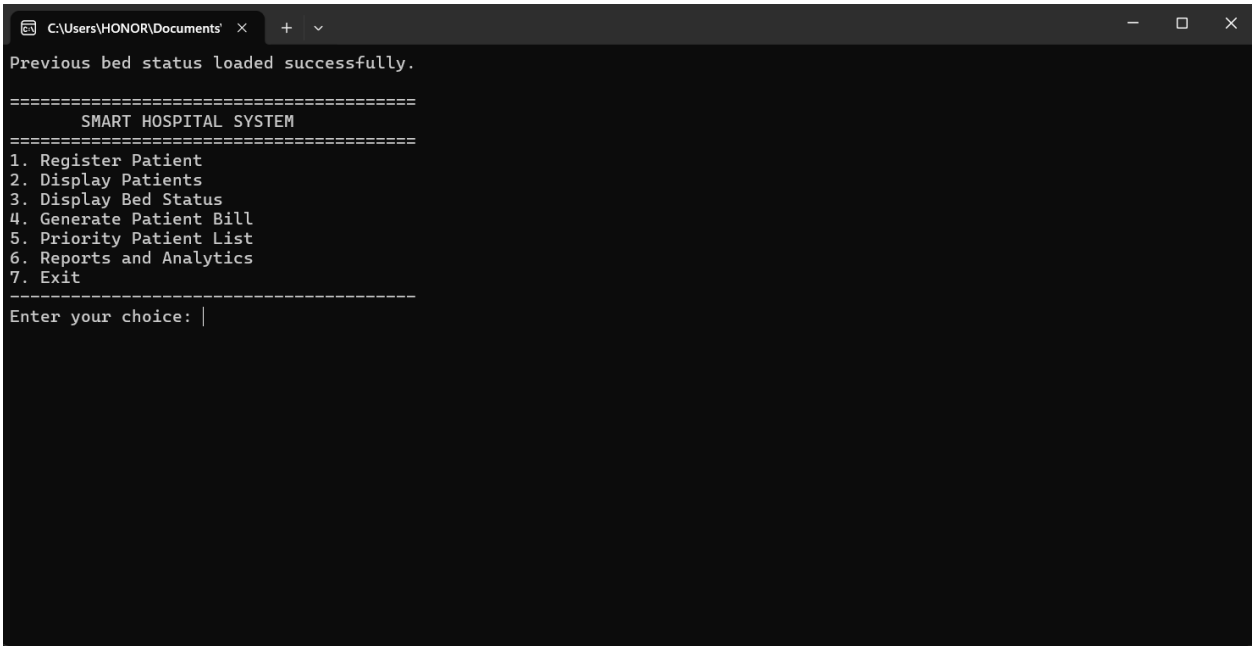
Main Menu

1. Register Patient
2. Display Patients
3. Display Bed Status
4. Generate Patient Bill
5. Priority Patient List
6. Reports and Analytics
7. Exit

The program uses separate functions for calculations and file handling while the main function controls the menu and system flow.

System Flow

Patient Registration → Data Validation → Waiting Time Calculation → Bed Allocation → Cost Calculation → Record Storage → Billing → Priority Sorting → Reports



4. Data Structures

The system mainly uses parallel arrays to store patient information.

Patient Arrays

```
char patientName[MAX_PATIENTS][50];
int patientAge[MAX_PATIENTS];
int patientEmergencyLevel[MAX_PATIENTS];
int patientSpecialtyID[MAX_PATIENTS];
int patientAdmitted[MAX_PATIENTS];
int patientWardID[MAX_PATIENTS];
```

int patientBedID[MAX_PATIENTS];
int patientDays[MAX_PATIENTS];
Additional arrays are used to store calculated values:
float patientWaitingTime[MAX_PATIENTS];
float patientSurcharge[MAX_PATIENTS];
float patientWardCost[MAX_PATIENTS];
The system supports up to 100 patients.
The assignment allows the use of parallel arrays instead of structures.

Bed Occupancy Matrix

A two-dimensional integer array is used to track hospital beds:
int bedOccupancy[4][20];
A value of 0 represents an available bed, while 1 represents an occupied bed.

5. Lookup Data

The system contains predefined lookup data for four medical specialties:

ID	Specialty	Consultation Fee	Consultation Time
1	General Practice	LKR 1,500	15 minutes
2	Paediatrics	LKR 2,500	20 minutes
3	Cardiology	LKR 4,500	30 minutes
4	Neurology	LKR 5,000	30 minutes

The system also contains four hospital wards:

ID	Ward	Daily Rate	Capacity
1	General Ward	LKR 3,000	20
2	Paediatric Ward	LKR 6,000	10
3	Surgical Ward	LKR 12,000	10
4	ICU	LKR 25,000	5

These values are based on the assignment specification.

6. Implemented Functions

6.1 Emergency Surcharge Calculation

The function calculateEmergencySurcharge() calculates an additional charge according to the patient's emergency level.

- Level 1 – Normal: 0%
- Level 2 – Urgent: 20%
- Level 3 – Critical: 50%

This follows the surcharge calculation specified in the assignment.

6.2 Money Formatting

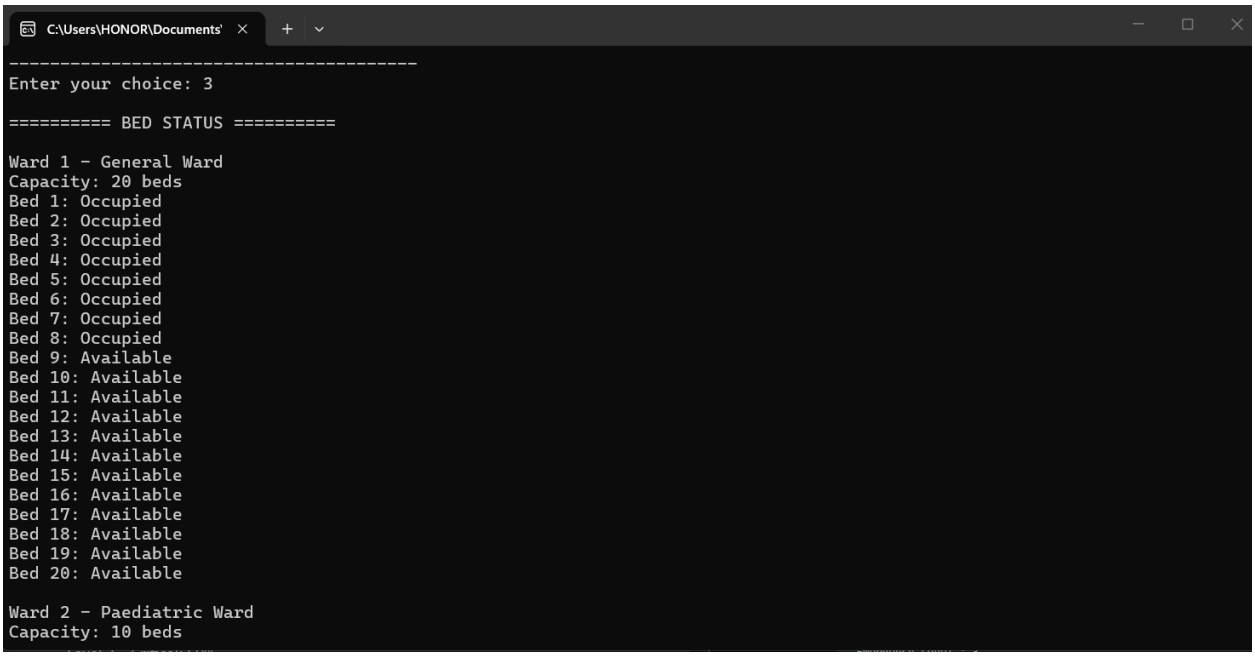
The printMoney() function formats financial values with commas and two decimal places.
For example:
LKR 82,500.00
This improves the readability of generated bills and reports.

6.3 Patient Record Storage

The savePatientRecords() function stores patient information in:
patient_records.txt
The stored information includes patient name, age, emergency level, specialty, admission status, ward, bed, admission days, waiting time, surcharge, and ward cost.
The system uses file append mode so newly registered patient information can be added to the existing record file.

6.4 Bed Status Storage

The saveBedStatus() function stores current bed occupancy information in:
beds_status.txt
The system records the ward number, bed number, and occupancy status.
The bed status can therefore be preserved between program executions.

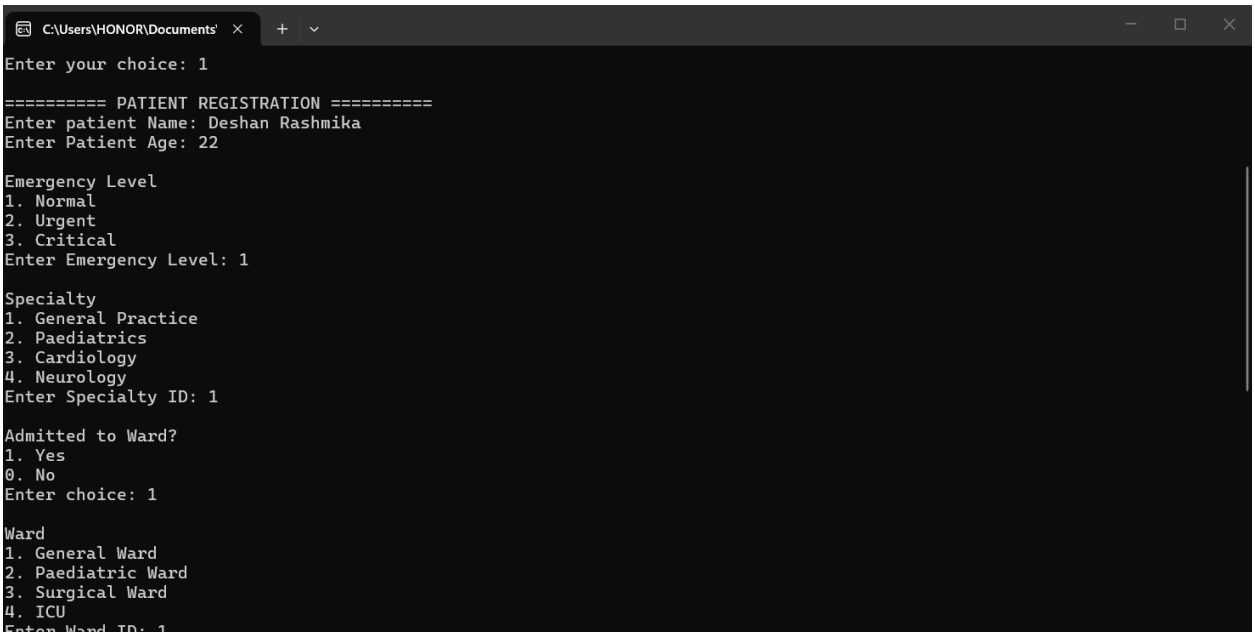


6.5 Patient Registration

The registration module collects:

- Patient name
- Patient age
- Emergency level
- Specialty ID
- Admission status
- Ward ID
- Number of admission days

The system also validates important inputs such as age, emergency level, and specialty ID.
These inputs correspond to the required patient intake information in the assignment.



6.6 Automatic Bed Allocation

When a patient is admitted to a ward, the system searches for an available bed.
When a free bed is found, the bed is assigned to the patient and its occupancy status is changed to occupied.
This prevents the same bed from being assigned to multiple admitted patients.

6.7 Waiting Time Calculation

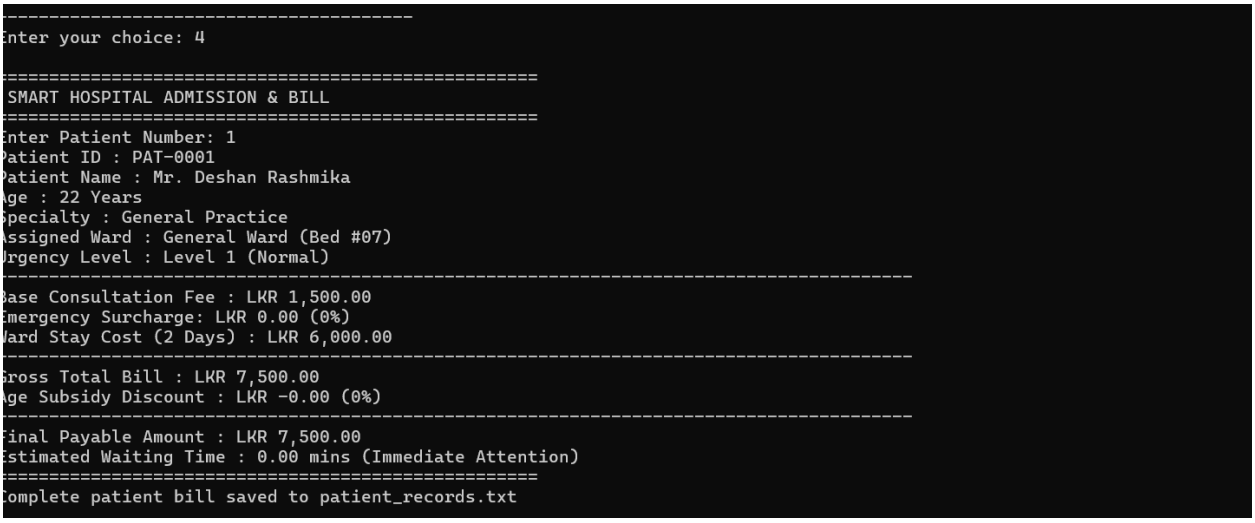
The estimated waiting time is calculated using the patient's selected specialty queue.
The assignment specifies:
Waiting Time = Current Queue Count × Average Consultation Time
After registration, the corresponding specialty queue count is increased.

6.8 Ward Stay Cost Calculation

For admitted patients, the ward cost is calculated using:
Ward Cost = Number of Days × Ward Daily Rate
For outpatient patients, the ward cost is set to zero.

6.9 Patient Bill Generation

The billing module calculates:
Gross Total =
Consultation Fee + Emergency Surcharge + Ward Stay Cost
An age subsidy is then applied.
Patients younger than 5 or older than 65 receive a 15% discount:
Discount = Gross Total × 15%
Finally:
Final Payable Amount = Gross Total − Discount
These calculations follow the assignment specification.
The complete billing information is also saved to patient_records.txt.



6.10 Priority Patient List

The Priority Patient List uses a sorting algorithm to arrange patients according to emergency level.
The priority order is:
Level 3 – Critical
Level 2 – Urgent
Level 1 – Normal
Patients with the same emergency level maintain their registration order.
This follows the assignment's priority requirements.

```
Enter your choice: 5

========== PRIORITY PATIENT LIST ==========

Priority 1
----------------------------
Name            : Deshan Rashmika
Age             : 22
Emergency Level : 1
Specialty ID    : 1
Waiting Time    : 0 minutes
Ward ID         : 1
Bed ID          : 7

Priority 2
----------------------------
Name            : Sadun
Age             : 70
Emergency Level : 1
Specialty ID    : 1
Waiting Time    : 15 minutes
Ward ID         : 1
Bed ID          : 8

========================================
```

6.11 Reports and Analytics

The Reports and Analytics module provides:

- Total number of patients
- Number of normal patients
- Number of urgent patients
- Number of critical patients
- Number of admitted patients
- Total revenue
- Total discounts
- Ward occupancy percentages
- Highest-paying patient

This provides a summary of the hospital's current patient and financial information.

```
--------------------------------------
Enter your choice: 6

========== REPORTS AND ANALYTICS ==========

----- Patient Statistics -----
Total Patients       : 2
Normal Patients      : 2
Urgent Patients      : 0
Critical Patients    : 0
Admitted Patients    : 2

----- Financial Statistics -----
Total Revenue        : LKR 13875.00
Total Discounts      : LKR 1125.00

----- Bed Occupancy -----
General Ward: 40.00% occupied
Paediatric Ward: 70.00% occupied
Surgical Ward: 50.00% occupied
ICU: 100.00% occupied

----- Highest-Paying Patient -----
Patient Name : Deshan Rashmika
Final Bill   : LKR 7500.00

========================================
```

7. File Handling

The system uses text files to maintain important information.

Patient Records

patient_records.txt

This file stores patient registration and billing information.

Bed Status

beds_status.txt

This file stores the current occupancy status of hospital beds.

File handling is implemented using standard C functions such as:

fopen()

fprintf()

fscanf()

fclose()

8. Input Validation

Input validation is included to reduce invalid data entry.

Examples include:

- Patient age must be between 0 and 120.
- Emergency level must be between 1 and 3.
- Specialty ID must be between 1 and 4.
- Ward ID must be within the available ward range.
- Invalid patient numbers are rejected during billing.
- Invalid menu input is handled without terminating the program.

9. Assumptions

The following assumptions were made during development:

1. A maximum of 100 patients can be stored during one program execution.
2. There are four medical specialties.
3. There are four hospital wards.
4. Bed capacities are fixed according to the assignment specification.
5. Emergency levels are represented by 1, 2, and 3.
6. Level 3 patients have the highest priority.
7. Waiting time is estimated using the current specialty queue count.
8. Patients below 5 years or above 65 years receive a 15% age subsidy.
9. Ward charges are calculated per admitted day.
10. Bed occupancy is represented using 0 for available and 1 for occupied.
11. Patient and bed records are stored using text files.
12. The system is a simulation and does not represent a real hospital billing system.

10. Testing

The system was tested using different patient scenarios.

Test Case 1 – Critical Patient

A critical patient was registered and admitted to the ICU.
Expected behaviour:

- Emergency Level = 3
- 50% emergency surcharge
- ICU bed allocated
- Ward cost calculated according to admission days
- Patient placed at the highest priority

Test Case 2 – Elderly Patient

An elderly patient above 65 years was registered.
Expected behaviour:

- 15% age subsidy applied
- Discount calculated from the gross bill
- Final payable amount reduced accordingly

Test Case 3 – Outpatient

A patient was registered without ward admission.
Expected behaviour:

- No bed allocated
- Ward cost = LKR 0.00
- Patient can still receive a consultation bill

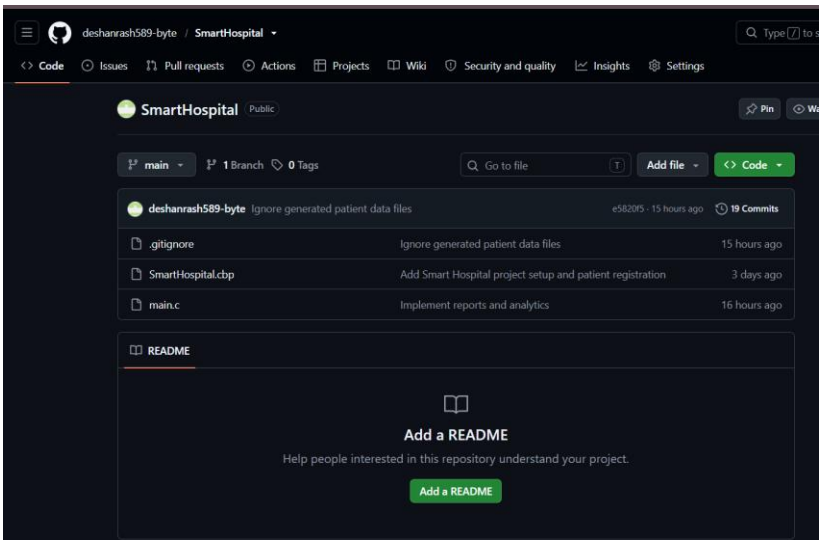
Test Case 4 – Multiple Patients

Multiple patients with different emergency levels were registered.
Expected behaviour:

- Patients were sorted according to emergency priority.
- Critical patients appeared before urgent patients.
- Urgent patients appeared before normal patients.

11. GitHub Repository

The project is maintained in a public GitHub repository as required by the assignment.
Repository:
https://github.com/deshanrash589-byte/SmartHospital
The repository contains the C source code and project files.
The project was developed incrementally using multiple meaningful commits throughout the development process.



12. Conclusion

The Smart Hospital & Resource Allocation System successfully implements the major requirements of the assignment using the C programming language.
The system demonstrates the practical use of parallel arrays, two-dimensional arrays, functions, loops, conditional statements, string handling, sorting algorithms, mathematical calculations, and file handling.
The completed system can register patients, allocate beds, calculate waiting times and medical charges, generate patient bills, organize patients according to emergency priority, maintain records, and produce hospital reports.
This project provided practical experience in applying fundamental C programming concepts to a real-world inspired resource allocation problem.